KANGANA AJAY KHURANA

B.Tech Computer Science & Engineering, SVNIT Surat • Surat, India
kanganakhurana04@gmail.com | github.com/some1iu | linkedin.com/in/PLACEHOLDER | [+91 XX XXX XXXXX]

PROFILE

I learn by building and by reading other people's code. Working independently I wrote and finished a Python + MySQL application end to end; as part of a four-person team I built client and server-side features for a full-stack events and expenses platform; on my own I got a large open-source monorepo running locally to study how it is put together. I want to move that habit — implement, measure, debug, repeat — into deep learning and reinforcement learning research engineering, and I am working through the linear algebra, probability and optimisation needed to implement learning algorithms from first principles rather than call a library.

SELECTED PROJECTS

Movie Streaming Platform *Python, MySQL*

Independent project • single module, ≈820 lines • github.com/some1iu/movie-streaming-platform

- Built a console application that lets a user register and log in, browse a movie catalogue by title, genre and release year, and rent or buy a title, with an itemised bill written to disk at the end of each transaction.
- Modelled the data in three tables and implemented every access path against them: `user` (username, name, password, three preferred genres, access level), `movie` (code, title, year, rating, genre, summary, rent and buy cost, binary asset) and `bill` (transaction number, date, rent/buy flag, movie code, user).
- Wrote a filter-driven query builder that appends WHERE clauses only for the criteria the user actually supplies — LIKE matching for titles, exact matching for genre and year — and used parameterised statements on the registration and search paths so user input is bound rather than concatenated.
- Generated recommendations by reading the user's three stored genre columns and building them into an IN query on the home screen; handled the receipt side with file handling, and stored movie image data as a BLOB that is written back out to a file when a transaction completes.
- Funnelled the risky call sites through one error handler covering `mysql.connector` errors, bad input, type and key errors, so an invalid entry returns the user to the menu instead of terminating the session. No automated tests yet; correctness was checked by exercising each flow by hand.

ACM Incursion — Conflict-Aware Events & Expenses Manager *Next.js, Express, Prisma, PostgreSQL*

Four-person team (client/server repository, not a solo build) • github.com/chetisha28/incursion-acm

- Contributed to a platform for ACM student chapters that detects venue, time and audience collisions between proposed events before publication, gates proposals behind peer endorsement, and tracks expenses against semester budgets.
- Implemented client-side components against the REST API — parsing the {success, message, data} envelope, surfacing 400/401/403/409 responses in the relevant form or access view, and keeping the session token in tab-scoped storage — and contributed to server-side pieces: JWT-protected routes, role-gated access across viewer, executive and core tiers, Zod request validation and `Decimal(12,2)` money columns in Prisma.
- Debugged integration faults across Prisma migrations, seed data and CORS configuration to get the client talking to the API locally, and verified the fixes with the repository's Jest/Supertest suite and its live end-to-end workflow script.

Open-Source Codebase Exploration — Typebot *Bun, Nx, Prisma, PostgreSQL*

Local setup and source reading only; no upstream contribution • github.com/some1iu/typebot.io

- Set up a large self-hosted chatbot-builder monorepo (apps/ and packages/ workspaces) locally to read how a production codebase of that size is organised: Bun and Nx workspace configuration, the package dependency graph, Prisma client generation and migration workflow, environment-variable plumbing across packages, and the PostgreSQL service the stack expects.
- Worked through the resulting dev-environment failures (Prisma client generation, environment variables, database connectivity) until the stack ran, using build output and startup errors to locate each fault.

TECHNICAL SKILLS

- **Python** — the language of my independent project: menu-driven control flow, SQL access through MySQL Connector/Python, file handling, shared error handling.
- **Databases** — MySQL schema design, joins and filtering, parameterised queries; PostgreSQL with Prisma (schema, migrations, seeding) in the team project.
- **Web and backend** — Express REST endpoints and JWT authentication, React/Next.js App Router client work, HTML/CSS, JavaScript at a basic level.
- **C and C++** — coursework level, including inheritance and compile-time versus runtime polymorphism.
- **Tools** — Git and GitHub, VS Code, Bun and Nx workspace commands, reading unfamiliar source trees.
- **Currently learning** — Node.js; NumPy and introductory Pandas; linear algebra, probability and optimisation. *(Learning, not project experience.)*

EDUCATION

B.Tech in Computer Science and Engineering — SVNIT, Surat *[Start Year] – Present*

Expected graduation [Month Year] • Relevant coursework: data structures and algorithms, discrete mathematics and probability, object-oriented programming, database management systems.

Senior Secondary (Class XII), CBSE — Delhi Public School, Pune *2024 – 2025*

Computer Science stream; the Python and MySQL project above was written for this course.